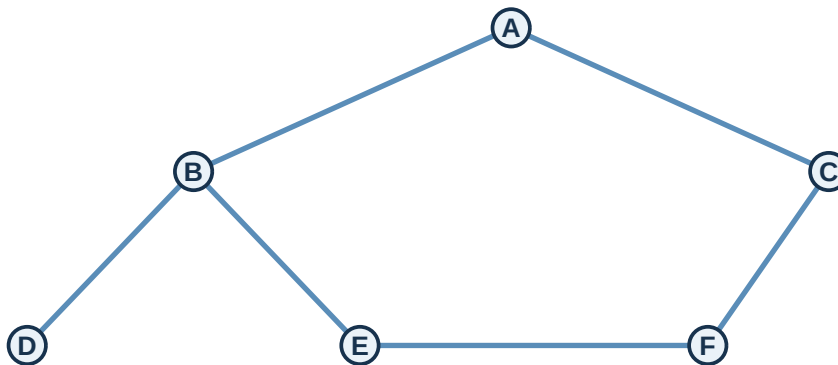


SOCIAL NETWORK GRAPH

Comparison of Adjacency Matrix and Adjacency List



Assignment Objective

Implement the given small social network using an adjacency matrix and an adjacency list. Perform BFS and DFS from vertex A, search for vertex E, check the edge E-F, and compare the two representations using execution results and asymptotic complexity.

Given connections: A-B, A-C, B-D, B-E, C-F, E-F

Vertices: A, B, C, D, E, F

Starting vertex: A **Search target:** E **Edge checked:** E-F

Implementation notes

The graph is undirected, so every connection is stored in both directions. The program processes neighbours in the order produced by the input connections; this fixes the traversal order used in the trace tables.

1. Graph Representations

The same network is represented in two different data structures. The matrix stores a value for every possible pair of vertices. The list stores only the neighbours that are actually connected.

1.1 Adjacency Matrix

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	1	0
C	1	0	0	0	0	1
D	0	1	0	0	0	0
E	0	1	0	0	0	1
F	0	0	1	0	1	0

A value of 1 means that the two vertices are connected; 0 means there is no direct connection. Because the graph is undirected, the matrix is symmetric.

1.2 Adjacency List

```
A -> B -> C -> NULL
B -> A -> D -> E -> NULL
C -> A -> F -> NULL
D -> B -> NULL
E -> B -> F -> NULL
F -> C -> E -> NULL
```

Observation

The graph has $V = 6$ vertices and $E = 6$ edges. The matrix reserves $6 \times 6 = 36$ cells, while the list stores one head for each vertex and two adjacency entries for each undirected edge, giving 12 adjacency entries.

2. BFS and DFS Traversal

Both traversals start at vertex A. The same neighbour ordering is used for the matrix and list versions so that the visit order can be compared directly.

2.1 Breadth First Search (BFS)

BFS uses a queue and explores the graph level by level.

Step	Removed	New vertices	Queue after step	Visited order
1	A	B, C	B, C	A, B, C
2	B	D, E	C, D, E	A, B, C, D, E
3	C	F	D, E, F	A, B, C, D, E, F
4	D	None	E, F	A, B, C, D, E, F
5	E	None	F	A, B, C, D, E, F
6	F	None	Empty	A, B, C, D, E, F

BFS order: A -> B -> C -> D -> E -> F

2.2 Depth First Search (DFS)

DFS follows one path as far as possible before backtracking.

Step	Current	Action	Traversal order
1	A	Visit B	A, B
2	B	Visit D	A, B, D
3	D	No unvisited neighbour; backtrack	A, B, D
4	B	Visit E	A, B, D, E
5	E	Visit F	A, B, D, E, F
6	F	Visit C	A, B, D, E, F, C
7	C	No unvisited neighbour; finish	A, B, D, E, F, C

DFS order: A -> B -> D -> E -> F -> C

3. Search and Edge-Checking Operations

For the execution comparison, vertex E is searched from A using a BFS-based search, and the existence of edge E-F is checked in both representations.

3.1 Search for Vertex E

Representation	Step	Processed vertex	Checks made	Result
Matrix	1	A	6	B and C discovered
Matrix	2	B	5	E found
List	1	A	2	B and C discovered
List	2	B	3	E found

Search result

Matrix search: E found after 11 neighbour checks.

List search: E found after 5 neighbour checks.

3.2 Edge Check for E-F

Representation	Vertex/list inspected	Checks	Result
Matrix	matrix[E][F]	1	Edge exists
List	E -> B -> F	2	Edge exists

The matrix checks one cell directly: matrix[E][F]. The list searches E's neighbours, which are B and F, so two list entries are inspected before finding F.

4. Execution Results and Complexity Analysis

The program was compiled with GCC using C11 and warning checks. The following counts are the actual neighbour checks produced by the implementation for the given graph.

4.1 Execution Comparison

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Storage for this graph	36 matrix cells	6 vertex heads + 12 adjacency entries
BFS	$O(V^2)$	$O(V + E)$
DFS	$O(V^2)$	$O(V + E)$
Edge checking	$O(1)$	$O(\text{degree}(u))$
BFS neighbour checks	36	12
DFS neighbour checks	36	12
Search for E from A	11	5
Edge check E-F	1	2

4.2 Complexity

Adjacency Matrix

Space: $O(V^2)$, because a $V \times V$ table is stored.

BFS: $O(V^2)$, since each visited vertex may scan all V matrix positions.

DFS: $O(V^2)$, for the same reason.

Edge checking: $O(1)$, because a single matrix cell gives the answer.

Adjacency List

Space: $O(V + E)$, because only vertices and existing edges are stored.

BFS: $O(V + E)$, because the traversal visits vertices and examines actual adjacency entries.

DFS: $O(V + E)$, for the same reason.

Edge checking: $O(\text{degree}(u))$ for a vertex u , with $O(V)$ as a general worst-case bound.

For this graph, $V = 6$ and $E = 6$. The traversal counts show 36 matrix neighbour checks versus 12 list neighbour checks, while the search for E required 11 matrix checks versus 5 list checks.

5. Final Comparison and Conclusion

The two representations provide the same graph and the same BFS/DFS orders when neighbours are processed consistently. Their main differences are in storage and the way adjacency information is accessed.

Criterion	Adjacency Matrix	Adjacency List
Storage	All possible vertex pairs are stored	Only existing connections are stored
Traversal work on this graph	36 checks for BFS and 36 for DFS	12 checks for BFS and 12 for DFS
Search for E	11 checks	5 checks
E-F edge check	1 direct check	2 list checks
Main advantage	Very fast direct edge test	Lower storage and efficient graph traversal

Final Conclusion

For a sparse social network, the adjacency list uses storage in proportion to the number of vertices and actual connections, rather than reserving a full $V \times V$ matrix. In this execution it also produced fewer neighbour checks for BFS, DFS and the search for E. The adjacency matrix remains useful when frequent direct edge-existence checks are required, as shown by its single-step E-F check. Based on the measured execution and the theoretical complexities, the adjacency list is the more space-efficient representation for this sparse graph, while the matrix provides the simplest constant-time edge test.

6. Submission Contents

The GitHub repository should contain: graph_comparison.c, input.txt, output.txt, README.md, report.md, and this PDF report (assignment_report.pdf).

Repository: social-network-graph-assignment

GitHub link: <https://github.com/calypso-gif/social-network-graph-assignment>

Source code and supporting files are provided separately in the repository so the execution can be checked against the report.